

Deep Learning Avancé & Applications

Transformers & Mécanismes d'Attention

Master Bac+5 – Data Analytics & Big Data
Mouaad AGOURRAM

2025–2026

Golden Collar

Des RNN aux Transformers

Mécanismes du Transformer

Bloc Décodeur

Les Trois Familles d'Architectures

Lois d'Échelle & Conclusion

Des RNN aux Transformers

Les Limites des RNN, LSTM et GRU

RNN vanilla — deux obstacles fondamentaux

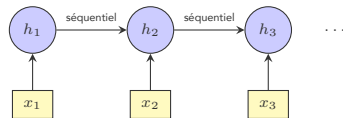
- **Séquentialité** : $h_t = f(h_{t-1}, x_t)$ — GPU sous-utilisé, complexité $\mathcal{O}(N)$
- **Vanishing Gradient** : BPTT multiplie des jacobiens — le contexte s'efface après 50+ tokens

LSTM / GRU — réponse partielle

- **LSTM** : Cell State c_t + 3 portes (oubli, entrée, sortie) — flux additif préserve le gradient sur de longues séquences
- **GRU** : 2 portes seulement (reset, update) — même idée, architecture plus légère

⇒ Le vanishing gradient est **résolu**

Limite résiduelle commune : malgré les portes, h_t dépend toujours de h_{t-1} — **la séquentialité persiste**. C'est ce verrou que le Transformer lève.



Impossible de paralléliser !

LSTM : c_t + 3 portes

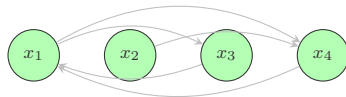
GRU : 2 portes (reset, update)

gradient préservé ✓

séquentialité maintenue ✗

Le changement de paradigme

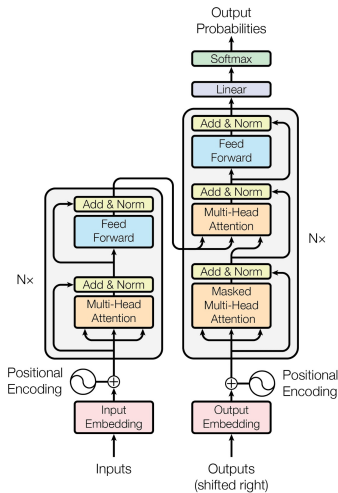
- **Éradication de la récurrence** : Remplacement par les mécanismes de *Self-Attention*
- **Parallélisation massive** : Traitement **simultané** de toute la séquence — complexité séquentielle réduite à $\mathcal{O}(1)$
- **Distance constante** : Connexion $\mathcal{O}(1)$ entre toute paire de tokens distants (x_i, x_j) — plus de dégradation due à la longueur du contexte



Tous les tokens en parallèle !

Contrepartie : l'attention est **quadratique** $\mathcal{O}(N^2)$ en longueur de séquence (mémoire + calcul) — d'où FlashAttention / Mamba (fin de séance).

L'Architecture Fondamentale du Transformer



Deux blocs principaux :

- **Encodeur** : Lit et comprend l'entrée complète en parallèle
- **Décodeur** : Génère la sortie token par token, guidé par l'encodeur

Composants clés de chaque bloc :

1. **Multi-Head Self-Attention** : Capture les relations entre tous les tokens
2. **Feed-Forward Network (FFN)** : Stocke la connaissance du monde
3. **LayerNorm + Connexions Résiduelles** : Stabilisent l'entraînement en profondeur

Source : "Attention Is All You Need", Vaswani et al., NIPS 2017

Pour désacraliser le flux mathématique, nous allons suivre cette phrase de bout en bout à travers chaque composant du Transformer :

"l'éléphant rose a essayé de monter dans la voiture, mais il était trop ..."

Étape 1

Tokenisation & Embedding

Étape 2

Self-Attention : qui regarde qui ?

Étape 3

FFN : quel fait est activé ?

Mécanismes du Transformer

1. Tokenisation et Projection Vectorielle

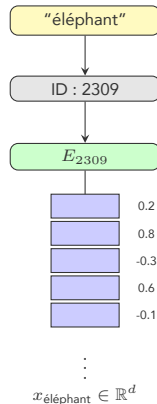
Pipeline pour : l' | éléphant | rose | a | essayé | de | monter |

...

1. **Tokenisation (BPE)** : Segmentation du texte en sous-mots
2. **ID mapping** : [64, 2309, 8740, 12, 4521, 65, 342, ...]
3. **Embedding** ($E \in \mathbb{R}^{|V| \times d_{\text{model}}}$) : Scalaire discret (ID) $\rightarrow$ vecteur dense sémantique x_i

Impact

"éléphant" mute en un vecteur continu encodant ses attributs sémantiques et sa proximité avec *animal* / *mammifère*.



2. Injection du Temps : Le Défi de l'Ordre

Puisque le Transformer traite toute la séquence **en parallèle**, il est nativement **aveugle à l'ordre grammatical**.

Problème

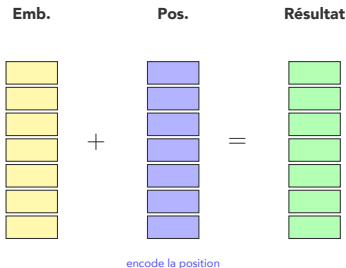
Sans signal positionnel, "Le chat mange la souris" et "La souris mange le chat" auraient la **même représentation vectorielle globale**.

Solution : Positional Encoding (PE)

On injecte un signal d'ordre dans chaque vecteur d'entrée :

$$X_{\text{input}} = E_{\text{batch}} + PE$$

Le vecteur **"éléphant"** en position 2 absorbe le signal PE_2 .



Encodage Absolu — Vaswani 2017

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Intuition : Comme les aiguilles d'une horloge tournant à différentes fréquences — chaque dimension code le temps continu.

La valeur est **additionnée** au vecteur d'embedding.

RoPE — Rotation Spatiale (Llama, Mistral)

Au lieu d'additionner, RoPE applique une **rotation** aux vecteurs Q et K dans le plan complexe :

$$q_m = q \cdot e^{im\theta}, \quad k_n = k \cdot e^{in\theta}$$

Propriété clé : Le produit scalaire d'attention

$$q_m \cdot k_n = x_m \cdot W_q \cdot R_{\theta, n-m} \cdot W_k^T \cdot x_n^T$$

$q_m \cdot k_n$ ne dépend que de $(m - n)$

⇒ meilleure généralisation sur les **longs contextes**.

Self-Attention : Principe de Base

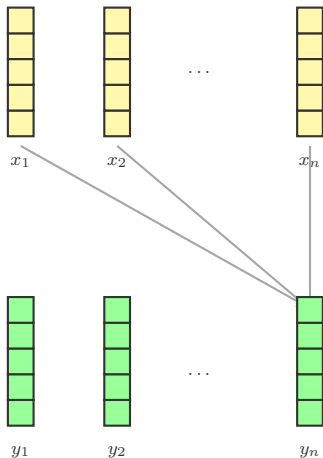
Chaque token de sortie y_i est une **combinaison pondérée** de tous les tokens d'entrée :

$$y_i = \sum_{k=1}^n a_{ik} x_k$$

avec $a_{ik} \geq 0$ et $\sum_{k=1}^n a_{ik} = 1$.

Intuition : Pour produire y_n , le modèle "regarde" tous les x_k et décide lesquels sont pertinents via les poids a_{ik} .

Ces poids sont **calculés dynamiquement** à partir du contenu — c'est la force du mécanisme d'attention.



Query (Q) — La Question

"De quoi ai-je besoin ?"

Le verbe **"monter"** génère une Query : *"Je cherche le sujet massif qui fait l'action."*

Key (K) — L'Étiquette

"Voici qui je suis."

Le mot **"éléphant"** déclare : *"Je suis une entité animale, imposante, sujet potentiel d'une action."*

Value (V) — Le Contenu

"Voici mon information."

Une fois le match Q/K établi, **"monter"** absorbe 95% du contenu d'**"éléphant"**.

Le Match Sémantique

La Query de **"monter"** rencontre la Key d'**"éléphant"** $\Rightarrow$ score ≈ 0.95 . **"monter"** n'est plus un verbe isolé, il devient le concept combiné de *l'action d'un éléphant rose*. C'est le **mélange de contexte**.

Mathématiques : Scaled Dot-Product Attention

Soit $X \in \mathbb{R}^{N \times d_{\text{model}}}$. Les 3 projections affines :

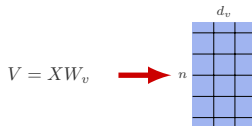
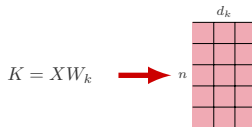
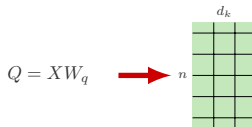
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Formule d'Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Pourquoi le facteur $\sqrt{d_k}$?

- À grande dimension, la variance de QK^T explose
- Le softmax sature sur ses plateaux asymptotiques $\Rightarrow$ gradient ≈ 0
- Diviser par $\sqrt{d_k}$ normalise la variance et préserve l'optimisation



$$Y = \text{softmax}\left[\frac{QK^T}{\sqrt{d_k}}\right] V$$

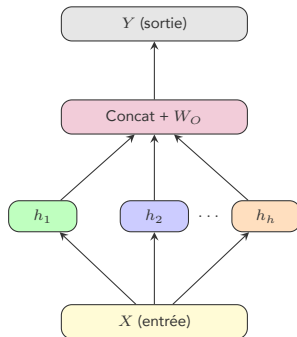
Multi-Head Attention : Factorisation des Sous-Espaces

Pourquoi h têtes parallèles ($h = 8, 16 \dots$) plutôt qu'une mega-matrice unique ?

Dissection sémantique parallèle

Un monobloc est contraint à un seul concept par couche. Diviser en sous-espaces ($d_k = d_m/h$) permet d'analyser **simultanément** des attributs orthogonaux :

- **Tête 1** : Dépendance Sujet $\leftrightarrow$ Verbe (éléphant/monter)
- **Tête 3** : Token $\leftrightarrow$ son Adjectif (éléphant/rose)
- **Tête 8** : Relations spatio-temporelles



Le Feed-Forward Network : Banque de Connaissances

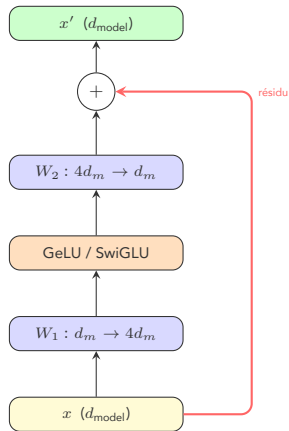
L'Attention est un **routeur dynamique** : elle indique quel token regarde quel autre token.

Mais **où** sont stockées les connaissances du monde ?

Le FFN comme Mémoire Associative

Le vecteur contextuel ("monter par un éléphant") traverse le FFN en 3 phases :

1. **Expansion** (W_1) : Projection $\times 4$ — crée l'espace de recherche
2. **Activation** (GeLU / SwiGLU) : Active les neurones porteurs du fait *"éléphant + lourd = trop grand pour une voiture"*
3. **Projection** (W_2) : Recompression à d_{model} + résidu



Layer Normalization : Formule et Paramètres Appris

Pipeline en 2 étapes

Étape 1 — Normalisation statistique (par token) :

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

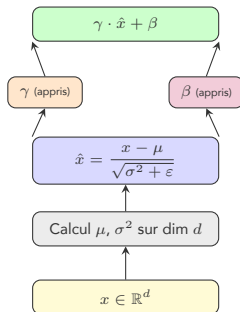
μ, σ^2 calculés sur les d_{model} dimensions du vecteur.

Étape 2 — Rescaling affine appris :

$$\text{LN}(x) = \gamma \cdot \hat{x} + \beta$$

γ et β sont des paramètres appris

- Initialisés à $\gamma = \mathbf{1}, \beta = \mathbf{0}$ (identité)
- Mis à jour par backprop comme n'importe quel poids
- Redonnent au modèle la liberté de **rescaler et déplacer** la distribution selon ce que la couche doit apprendre
- **Connexion résiduelle (+)** : le signal d'origine court-circuite le sous-bloc — out = $x + f(x)$ — préserve le gradient et évite l'annihilation sur des dizaines de couches



Architecture du Bloc Encodeur (Pre-LayerNorm)

Les trois briques vues jusqu'ici s'assemblent en un seul bloc :

LN	normalise avant chaque sous-bloc
MHA (bidi)	chaque token voit toute la séquence
FFN	injecte les connaissances mémorisées
(+)	résidu préserve le signal original

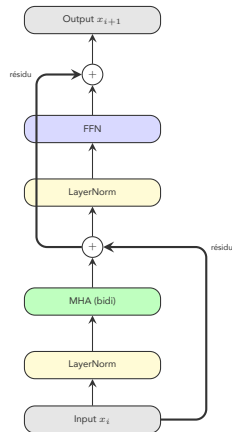
Formule du bloc :

$$x' = x + \text{MHA}(\text{LN}(x))$$

$$x'' = x' + \text{FFN}(\text{LN}(x'))$$

En sortie

N blocs empilés $\Rightarrow$ pour chaque token, un vecteur qui encode sa signification dans le contexte **complet** de la séquence.



Bloc Décodeur

Interdire l'Oracle : Le Masquage Causal

Next Token Prediction : lors du pré-entraînement génératif, sur le token "**monter**", le modèle doit prédire "**dans**".

Sans contrainte, Q_{monter} pourrait s'aligner sur les tokens **futurs** — une **triche oracle** qui corrompt les poids appris.

La Matrice de Masquage Causal

On annule chaque logit futur avec $-\infty$:

$$M_{ij} = \begin{cases} 0 & \text{si } i \geq j \\ -\infty & \text{si } i < j \end{cases}$$

Après softmax : $\lim_{x \rightarrow -\infty} e^x = 0$.

Tout token futur devient **invisible**.

Matrice d'attention masquée

t_1	a_{ij}	$-\infty$	$-\infty$	$-\infty$
t_2	a_{ij}	a_{ij}	$-\infty$	$-\infty$
t_3	a_{ij}	a_{ij}	a_{ij}	$-\infty$
t_4	a_{ij}	a_{ij}	a_{ij}	a_{ij}
	t_1	t_2	t_3	t_4

visible

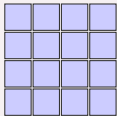
masqué ($-\infty$)

Encodeur vs Décodeur : Deux Régimes d'Attention

Self-Attention Encodeur — Bidirectionnelle

Chaque token voit **tous** les autres tokens :

$$A_{ij} = \text{softmax}\left(\frac{q_i \cdot k_j}{\sqrt{d_k}}\right), \quad \forall i, j$$

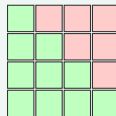


matrice pleine — comprendre

Self-Attention Décodeur — Causale

Chaque token ne voit que **sa gauche** :

$$A_{ij} = \begin{cases} \text{softmax}(\cdot) & j \leq i \\ 0 & j > i \end{cases}$$



triangulaire — générer

Même formule $\text{softmax}(QK^T / \sqrt{d_k})V$ — seul l'**ajout du masque** M distingue les deux régimes

Architecture du Bloc Décodeur Moderne (Pre-LayerNorm)

Même structure que le bloc encodeur — **un seul changement.**

MHA bidirectionnelle → MHA causale

Le masque causal (slide précédent) est appliqué à l'intérieur de la MHA :

$$A_{ij} = 0 \text{ si } j > i$$

Chaque token ne voit que son passé — génération autorégressive.

Formule identique :

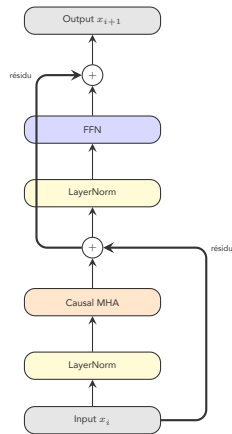
$$x' = x + \text{MHA}_{\text{causal}}(\text{LN}(x))$$

$$x'' = x' + \text{FFN}(\text{LN}(x'))$$

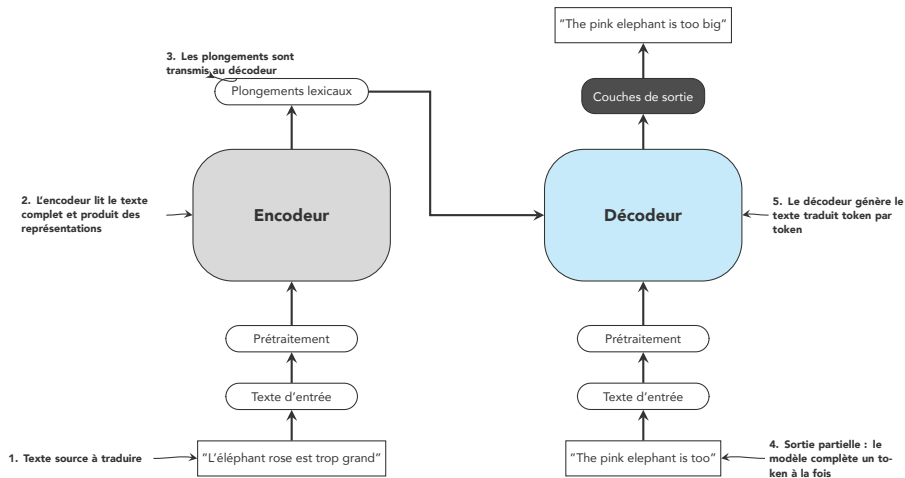
Pourquoi "Moderne" ?

Post-LN (Vaswani 2017) : $x' = \text{LN}(x + \text{MHA}(x))$ — LN après la somme

Pre-LN (GPT-2, LLaMA) : $x' = x + \text{MHA}(\text{LN}(x))$ — LN avant, plus stable à grande échelle



Vue d'Ensemble : Flux Encodeur → Décodeur



Les Trois Familles d'Architectures

Trois Architectures, Trois Philosophies

Le Transformer 2017 est un **Encodeur-Décodeur** complet. La communauté l'a décliné en trois familles spécialisées :

Encodeur seul — Comprendre

Attention **bidirectionnelle**

BERT · RoBERTa · DeBERTa

Décodeur seul — Générer

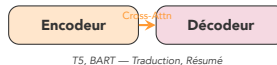
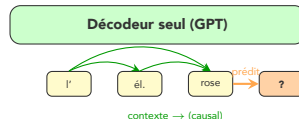
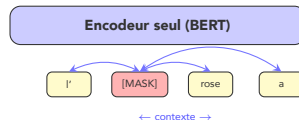
Attention **causale** (gauche → droite)

GPT · LLaMA · Mistral · Nemotron

Encodeur-Décodeur — Transformer

Lecture complète + génération guidée

T5 · BART · mT5 · PEGASUS



Encoder-Only : Pré-entraînement et Tâches

Tâche 1 — Masked Language Modeling (MLM)

15% des tokens remplacés par [MASK] — le modèle prédit le token masqué en s'appuyant sur le contexte **gauche ET droite**.

Tâche 2 — Next Sentence Prediction (NSP)

L'input est une **paire de phrases** séparées par [SEP].
Le token [CLS] prédit : *la phrase B suit-elle la phrase A ?*
⇒ force le modèle à apprendre la **cohérence inter-phrases**.

Nuance : RoBERTa (2019) montrera que NSP n'apporte rien et l'abandonne.

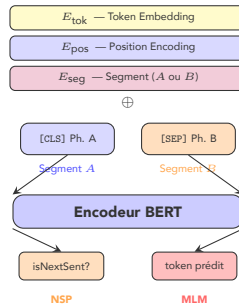
3 couches d'embedding additionnées : $x_i = E_{\text{tok}} + E_{\text{pos}} + E_{\text{seg}}$

- E_{tok} : représentation sémantique du token
- E_{pos} : position dans la séquence
- E_{seg} : appartenance à la **Phrase A** ou **Phrase B**

Tâches : Classification · NER · QA extractif · Embeddings

BERT, RoBERTa, CamemBERT, DeBERTa, ALBERT

Input x_i = somme de 3 embeddings :



BERT : Tokens Spéciaux & Fine-Tuning

Tokens spéciaux : [CLS] et [SEP]

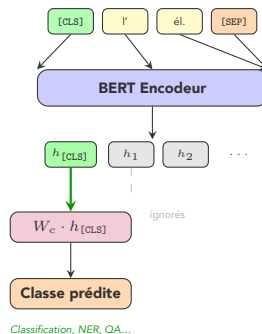
[CLS] (1^{er} token) : son vecteur de sortie $h_{[CLS]}$ agrège le sens de toute la séquence — utilisé comme **représentation globale** pour la classification.

[SEP] : marque la frontière entre deux phrases (NSP) ou la fin de séquence.

WordPiece : tokenizer de BERT — "running" → ["run", "##ing"] selon les fréquences de sous-mots appris.

Fine-Tuning : 3 étapes pour une tâche cible

1. **Pré-entraîner** sur MLM + NSP (non supervisé, 1 fois)
2. **Ajouter** une tête linéaire W_c sur $h_{[CLS]}$:
 $\hat{y} = \text{softmax}(W_c \cdot h_{[CLS]})$
3. **Fine-tuner** tout le modèle sur données étiquetées (quelques heures)



Decoder-Only : Pré-entraînement et Tâches

Pré-entraînement : Next Token Prediction

1. Le modèle reçoit une séquence et prédit **le prochain token**
2. **Masquage causal** : chaque token ne voit que sa gauche
3. GPT-3 : 300B tokens — CommonCrawl 60 %, Books 16 %, Wikipedia 3 %

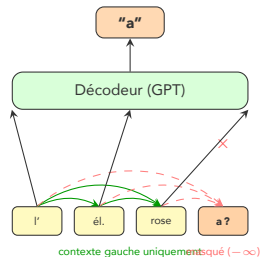
Comportements Émergents (non programmés !)

Par simple prédiction, le modèle apprend à **traduire**, **raisonner** (CoT) et **programmer**.

Tâches de prédilection :

- **Génération** de texte libre (articles, récits)
- **Complétion de code** (GitHub Copilot)
- **Chatbot / assistant** (ChatGPT, Claude)
- **Résumé** autoregressif

Modèles phares : GPT-3/4, LLaMA, Mistral, Nemotron, Falcon



Encoder-Decoder : Pré-entraînement et Tâches

Cross-Attention : le pont Encodeur → Décodeur

Le décodeur ajoute un 3^e sous-bloc entre ses deux MHA : la **Cross-Attention**.

Q vient du décodeur K, V viennent de l'encodeur

⇒ le décodeur peut "lire" la séquence source à chaque token généré.

Pré-entraînement : Corruption & Reconstruction

T5 (Text-to-Text Transfer Transformer) :

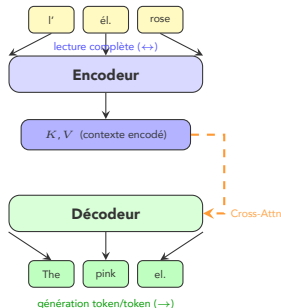
- Des *spans* de tokens sont masqués (**Span Corruption**)
- Le décodeur doit reconstruire les spans manquants

BART :

- Input fortement corrompu (shuffle, masquage, bruit)
- Le décodeur **débruite** et reconstitue le texte original

Tâches de prédilection :

- **Traduction automatique** (haute précision)
- **Résumé de texte** (compréhension complète de la source)
- **QA génératif** (produire une réponse, pas l'extraire)



Synthèse : Quelle Architecture pour Quelle Tâche ?

Critère	Encodeur seul	Décodeur seul	Enc-Déc
Attention	Bidirectionnelle	Causale	Bi + Causale
Pré-entraînement	MLM (masquage)	Next Token Pred.	Dénoisement / Seq2Seq
Représentation	Contextualisée	Autorégressive	Source + Cible
Classification	✓✓✓	✓	✓
NER / Extractif	✓✓✓	✓	✓
Génération libre	—	✓✓✓	✓✓
Traduction	—	✓	✓✓✓
Résumé	—	✓✓	✓✓✓
Chatbot / Assistant	—	✓✓✓	✓

Encodeur — Analyser

Idéal pour **extraire** et comprendre du texte existant.

Décodeur — Créer

Idéal pour **produire** du texte de haute qualité.

Enc-Déc — Transformer

Idéal pour les tâches **séquence-à-séquence**.

Lois d'Échelle & Conclusion

Lois d'Échelle & Propriétés Émergentes

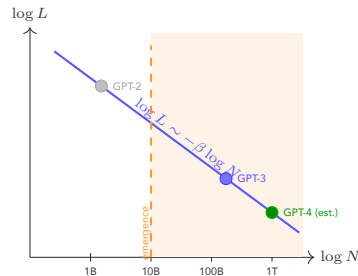
Lois d'Échelle (Kaplan 2020 · Hoffmann 2022)

En espace **log-log**, la perte suit une droite de pente $-\beta$:

$$\log L \approx -\beta_N \log N + c_N$$

Chinchilla (2022) : tokens optimaux $D^* \approx 20 N$ — modèle *compute-optimal*.

- **Émergences** (Wei et al. 2022) : au-delà de $\approx 10^{10}$ paramètres, des capacités **non programmées** apparaissent — *in-context learning*, *Chain-of-Thought*, suivi d'instructions.
- **Nuance** (Schaeffer et al. 2023) : certaines « émergences » sont un **artefact de métriques discontinues** (exact-match) ; avec une métrique lisse, la courbe redevient continue.



Courbe illustrative ; la taille de GPT-4 n'est pas publique (estimation).

Compute-optimal (Hoffmann et al., 2022)

À budget de calcul fixe ($C \approx 6ND$ FLOPs), il existe un **équilibre optimal** entre la taille du modèle N et le nombre de tokens D :

$$N^* \propto C^{0.5}, \quad D^* \propto C^{0.5} \Rightarrow D^* \approx 20 N^*$$

GPT-3 était sous-entraîné

Chinchilla 70B (1.4 T tokens) **bat** **GPT-3 175B** (300 B tokens) : à calcul égal, mieux vaut **plus de données** qu'un modèle plus gros.

Décomposition de la perte

$$L(N, D) = \underbrace{E}_{\text{borne irréd.}} + \underbrace{\frac{A}{N^\alpha}}_{\text{modèle trop petit}} + \underbrace{\frac{B}{D^\beta}}_{\text{trop peu de données}}$$

On réduit la perte en augmentant N **et** D **de concert**.

Conséquence pratique (2023+)

La **donnée de qualité** — et non la seule taille — devient le goulot. D'où LLaMA / Mistral : modèles plus **petits**, entraînés **bien plus longtemps**.

Dépasser le coût $\mathcal{O}(N^2)$ de l'attention

- **FlashAttention** : attention exacte, IO-aware, mémoire $\mathcal{O}(N)$
- **Mamba / SSM** : récurrence linéaire $\mathcal{O}(N)$ pour les longues séquences
- **Attention sparse / locale** : Longformer, sliding window (Mistral)

Longs contextes

Position interpolation / RoPE scaling $\Rightarrow$ 100k+ tokens ;
KV-cache pour une génération incrémentale efficace.

Capacité & modalités

- **Mixture-of-Experts** (Mixtral) : paramètres **sparse**, calcul constant
- **Multimodal** : ViT, CLIP, modèles vision-langage

Alignement (post-training)

Un modèle pré-entraîné n'est pas un assistant : **RLHF** et **DPO** l'alignent sur les préférences humaines.

Ce qu'on doit retenir de cette séance

Le mécanisme

- **Self-attention** remplace la récurrence : tout en parallèle, distance $\mathcal{O}(1)$ entre tokens — au prix d'un coût $\mathcal{O}(N^2)$
- $\text{softmax}(QK^T / \sqrt{d_k}) V$: le $\sqrt{d_k}$ évite la saturation du softmax
- **Multi-head** : sous-espaces parallèles ; **FFN** : mémoire associative ; **LayerNorm + résidu** : stabilité en profondeur

Position & génération

- Encodage positionnel : sin/cos (absolu) vs **RoPE** (relatif, longs contextes)
- **Masque causal** $\Rightarrow$ génération autorégressive (décodeur)

Trois familles

- **Encodeur** (BERT) : comprendre — attention bidirectionnelle
- **Décodeur** (GPT, LLaMA) : générer — attention causale
- **Enc-Déc** (T5, BART) : transformer — cross-attention

Échelle & efficacité

- Lois d'échelle (Chinchilla $D^* \approx 20N$) + capacités émergentes
- Efficacité : **FlashAttention**, **KV-cache**, **Mamba** (SSM)